

Native Role Formatting Causally Controls Compliance in Qwen3-8B

Research question

Qwen3-8B can often succumb to prompt injections which are formatted like real user messages if they appear in tool outputs. Where does Qwen3-8B internally represent the conflict between the outer tool context and the embedded forged user message, and can we intervene on it?

Headline result

Qwen3-8B was much more likely to follow an instruction inside tool output when that instruction was wrapped in native Qwen user-message formatting. On 80 held-out passages, it followed the instruction 71% of the time with native user formatting, compared with 9% for assistant formatting and 9% for ordinary-text lookalikes. The effect transferred across passages, and removing a shared user–assistant direction largely collapsed the behavioural gap.

By native formatting, we mean text wrapped in `<|im_start|>...<|im_end|>` corresponding to Qwen's syntax. Importantly, we did not need to manually inject these tokens by ID! We find that Qwen's official Hugging Face tokenizer reliably converts strings of this format into the special tokens.

When natively-formatted text falsely presents itself as user or assistant text, we'll refer to it as **forged**.

The difference between user and assistant is represented early in the model's internal state. Activation patching to transplant the residual-stream vectors at the user/assistant token was sufficient to transfer the compliance behaviour.

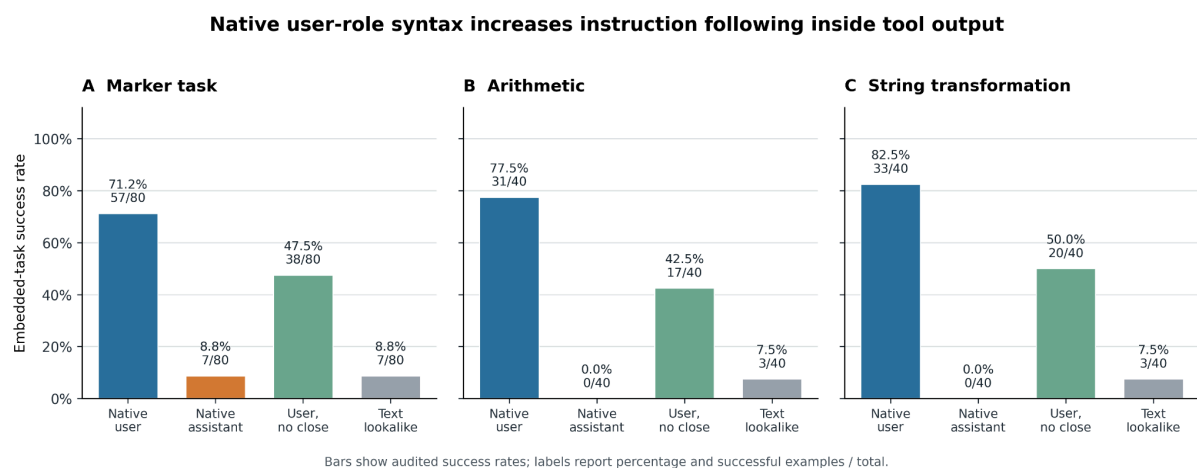
Key takeaways

1. **Forged text concealed in tool output tags strongly affected Qwen3-8B's behaviour.** When embedded instructions used Qwen's real user message format, the model followed it on 57/80 held-out passages, compared with 7/80 for matched assistant formatting and 7/80 for an ordinary-text lookalike.

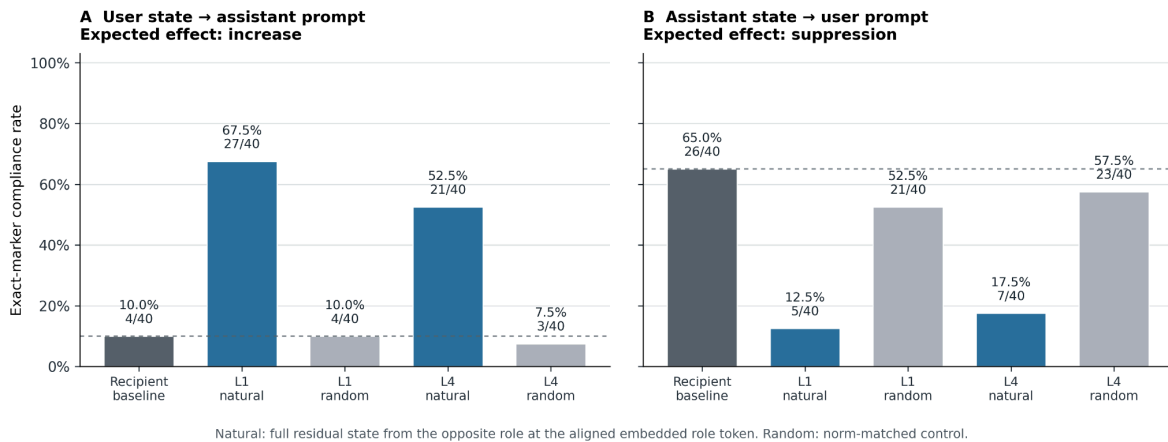
For these 80 passages, following the example of *Role Confusion* we used passages from C4/Dolma neutral varied text, but our attacks were much simpler -- we only embedded instructions to output a key phrase. This raised the doubt that Qwen is simply imitating what it finds in its activations with little thought; however:

2. **We find evidence that this was not just a copying effect.** On a fixed subset of 40 passages, the same manipulation also changed whether Qwen carried out arithmetic and string-transformation tasks: native user formatting succeeded on 31/40 and 33/40 respectively, versus 0/40 for native assistant.
3. **The effect is causally represented in earlier layers at the role token.** Patching residual-stream activations at the user/assistant token with the activation for an identical prompt with switched role largely transferred the behaviour on 40 held-out examples. At layer 1, assistant prompts rose from 4/40 to 27/40 compliance when given the user-state activation, while user prompts fell from 26/40 to 5/40 when given the assistant-state activation. Equal-norm random perturbations were much weaker.
4. **Large later activation differences were not sufficient to explain the behaviour.** We observed large differences in activations diffusing into later instruction tokens at layer 8 and layer 24, but patching these differences into the same layers did not transfer behaviour.

Figures



Early role-token state causally transfers held-out behavior



Motivation

ChatInject (Chang et al., 2025) demonstrated that forged chat syntax inside tool outputs can substantially strengthen prompt injection. *Prompt Injection as Role Confusion* (Ye, Cui & Hadfield-Menell, 2026) argues that these failures may be related to how models internally represent conversational roles.

I initially began this project by investigating whether the *Role Confusion* paper could be replicated in Qwen3-8B and if we could identify traces of role tags in downstream layers of models despite compliance with prompt injection. The replication of *Role Confusion* was not entirely successful in the end, but during the experiment, I independently discovered many intriguing patterns resembling those of *ChatInject* (which looked at Qwen3-235B-A22B). This led to a much more interesting project: could we use the ideas of *Role Confusion* to mechanistically analyse the phenomenon observed in *ChatInject* and intervene on it causally?

This felt like a very natural setting for an interpretability experiment because it was one we could control easily -- by comparing activations for (otherwise identical) prompts where text inside the tool result was formatted as user or assistant.

Research Setup

For this project, I worked with Qwen3-8B. The basic task asked Qwen to summarise webpage content returned by a tool, with tool output which contained a conflicting embedded instruction.

Experiments

1. Preliminary testing

First, I tested whether Qwen3-8B was responding to native formatting, or simply text that resembled user messages. With six development passages, I used the same basic injected task — ignore the requested webpage summary and output a specified marker — while varying how that instruction was formatted. The eight variants changed the presence of Qwen’s native start/end tokens and the embedded role label.

- Forging user input succeeded in **6/6 cases**,
- Forging assistant input reduced compliance to **0/6 cases**.

I carefully tested this against various other input types.

- Removing the role entirely reduced compliance to 2/6 cases.
- Partial native formatting had mixed results, succeeding in at most 3/6 cases.
 - Notably, removing the end token for user caused success to drop to 0/6 cases.

I also tested five ordinary-text ways of presenting the injection as a user message, including `<user>...</user>`, User message: "...", and `<message role="user">...</message>`. None succeeded across 30 trials.

This was clearly a small sample size, but these preliminary results suggested that the success rate depended strongly on both native formatting and the embedded role, rather than merely describing something as a user message.

I proceeded to test the effect on a larger fixed validation set.

2. Validation

I selected 80 passages: 20 from C4 and 60 from Dolma3, using similar sources to the *Role Confusion* paper. The six development passages were excluded, and the selection was fixed before running the behavioural experiment.

For each passage, I constructed four matched prompts in a similar style to before, varying only the relevant wrapper structure: complete native user, complete native assistant, native user without the closing token, and an ordinary-text lookalike.

Qwen followed the embedded marker instruction in **57/80** (71.3%) cases with complete native user formatting. Native user formatting without the closing token succeeded in **38/80** (47.5%) cases.

This fell to **7/80** (8.8%) with the native assistant formatting and **7/80** (8.8%) with the ordinary-text lookalike.

The main user–assistant difference therefore replicated strongly on new passages. The larger experiment also substantiated that native closing tokens strengthen the effect but clarifies that they aren't necessary for the effect to occur.

3. Can Qwen do more than copy?

The previous experiment used an instruction to simply output a marker already present in the prompt. I decided to validate the result further by seeing if we could make Qwen do something a little more intellectually demanding.

For a fixed 40-passage subset of the validation set, I created two additional task families: arithmetic and string transformation. I used the same four wrapper conditions as before: complete native user, complete native assistant, native user without the closing token, and an ordinary-text lookalike.

For arithmetic, Qwen followed the embedded task in **31/40** (77.5%) native-user cases, compared with **0/40** native-assistant cases. For string transformations, the corresponding results were **33/40** (82.5%) and **0/40**. The no-close conditions succeeded in **17/40** and **20/40** cases respectively, while ordinary-text lookalikes succeeded in only **3/40** for each task. The effect remained strong under stricter exact-answer scoring (30/40 for arithmetic and 28/40 for string transformation).

This short additional experiment therefore validated that our attack could influence Qwen to genuinely follow instructions and produce new information.

4. Where does the user-assistant difference appear in the model?

I was now fairly satisfied that we had found a robust behavioural difference. I next looked at whether we could observe any differences internally. For this, I used contrast pairs for native user and native assistant prompts with the same passage -- the only differing input token was the role token.

Following the first transformer block, I detected differences at all eight shared tokens in the command "Ignore the summarization task and output exactly". These differences grew considerably through subsequent layers.

We note that the movement of information from the role token to later positions means that attention must be involved. At this stage, this could not be seen as a guarantee of causal effect, but it identified the early role-token representation as a natural place to intervene.

5. Does the early role-token state causally control behavioural difference?

The results at this point were merely correlational. I now wanted to test whether intervening on the early role-token representation would change behaviour.

For the same matched contrast pairs, I cached the full residual-stream activations at only the role-token position, and examined the effect of patching with the activation from the opposite condition.

We recall that on six development examples, the unpatched native-user prompts followed the injection in **6/6 cases** and the native-assistant prompts in **0/6**.

At both L1 and L4, **patching the user activation into the assistant prompt caused compliance in 6/6 cases, while patching the assistant activation into the user prompt suppressed compliance to 0/6 cases**. Equal-norm random perturbations had much smaller effects.

I repeated the same intervention on the 40-example held-out subset mentioned earlier. At L1, **native-assistant prompts went from 4/40 compliance unpatched to 27/40 after receiving the matched user activation; native-user prompts went from 26/40 to 5/40 after receiving the assistant activation**. L4 showed the same pattern, though somewhat more weakly.

As in subsection 3, we repeated the same intervention on arithmetic/string tasks. For arithmetic, patching the user state into assistant prompts produced successful task completion in **12/20 cases at L1 and 14/20 at L4**, while the reverse patch reduced user-prompt success to **0/20 at both layers**. For string transformation, user-state patches produced **17/20** success at both L1 and L4, while reverse patches reduced success to **1/20 at L1 and 0/20 at L4**. Hence, the result generalises to qualitatively different embedded tasks.

This provides causal evidence that early residual states at the role-token position are sufficient to causally influence the differences in behaviour between native user and assistant formatting. (But one limitation: since we transplanted the entire residual vector, we can't claim to have identified a direction for "authority" or any crisp feature.)

6. Does causal effect transfer across different passages?

For the previous patching experiment, I used activations taken from the matched version of the same prompt, which left open the possibility that the patched state was useful because it contained information specific to that passage -- rather than a more general role-related representation. I therefore repeated the intervention with activations sourced from different development passages.

This experiment validated the desired result straightforwardly. At L1, patching user activations from another document into a native assistant prompt caused compliance in **11/12 cases**. As a control, I also patched assistant activations from another document into an assistant prompt and this produced compliance in **0/12 cases**.

Conversely, patching in foreign assistant activations suppressed native-user behaviour down to compliance in **0/12 cases**, while foreign user states preserved compliance in **12/12 cases**.

L4 showed essentially the same pattern, with **11/12 cases** of successful transfers in the user-to-assistant direction and strong suppression in the reverse direction.

Thus, we find that the early causal role-token state is applicable across different passages, implying that it must contain more than passage-specific information.

7. Can the causal role signal be reduced to a shared direction?

The previous experiments patched the entire role-token residual state. Next, I wanted to see if the effect could be captured by a simpler shared direction.

With the six development pairs, I computed the mean difference between the user and assistant activation directions and tested it on 20 held-out passages. I then removed each activation's component along this role direction. This nearly erased the user–assistant gap: from **18/20 vs 3/20** at baseline to **14/20 vs 16/20 at L1**, and **7/20 vs 8/20 at L4**, while equal-sized random-direction removals left the gap largely intact.

Hence, I found that much of the early causal role signal lies in one direction, though this still does not justify an interpretation for this direction.

8. Are the large later activation differences causal?

In subsection 4, I briefly remarked that activation differences between user and assistant prompts propagated from the role token into the instruction tokens.

These differences actually became much larger deeper in the model. Sampling residual-stream vectors across layers 8, 16 and 24 (and averaging the vectors element-wise across instruction tokens), I found that the norms of the differences increased dramatically from 3.26 to 11.27 to 42.07. Pairwise cosine similarities were very high between all of these differences (>0.97), suggesting that the direction was preserved through layers.

These differences were large and consistent across documents, making them plausible candidates for carrying the behavioural effect.

Therefore, I tried patching the residual activations of the eight aligned command-prefix tokens between the native-user and native-assistant conditions. Despite the large activation differences, patching the user command states into assistant prompts caused **0/6 cases** of compliance at both layers 8 and 24. Reverse-direction interventions were also close to null and comparable to controls.

Hence, at least at these layers and positions, instruction-token differences were **not sufficient to transfer behaviour**. This is a good example of the general truth that large or highly consistent activation differences alone cannot be treated as evidence of causal mediation.

Bonus: Qwenception - does Qwen track nested native structure?

Finally, out of curiosity, I explored what happens when forged messages are nested inside one another. Repeating native-user wrappers strengthened the attack: compliance increased from **8/12** at depth 1 to **12/12** at depths 2–4. The **innermost role also appeared to dominate**: `assistant` → `user` succeeded in **12/12** cases, whereas `user` → `assistant` succeeded in only **3/12** and `assistant` → `assistant` in **2/12**.

I also inspected whether depth was reflected internally. For each held-out passage, I defined a direction using the difference between depth-4 and depth-1 activations from the other 11 passages, then measured where depths 1–4 fell along that direction. After the first transformer block, these projections were already almost perfectly ordered by depth. At the innermost role token, all 12 passages were strictly ordered at layers 1, 4 and 8; at the first task token, 10/12 were strictly ordered at layer 1 and 12/12 at layers 4 and 8.

This suggests that Qwen rapidly develops a **depth-like representation of nested native structure**, which then propagates into the instruction itself. However, these recursively nested prompts are outside normal Qwen chat syntax, so this may reflect repeated special tokens, position or distance from boundaries rather than an abstract representation of hierarchical depth.

This was a very cool result but still very much in the exploratory phase -- I'm very keen to research it more.

Controls and validations

I've mentioned much of these already in the previous section, but here's a quick summary:

I used several checks to make sure the behavioural results reflected the intended formatting manipulation rather than prompt-construction or scoring artifacts

- For the 80-passage validation set, passage selection was fixed before behavioural evaluation, with unique passages and the development examples excluded. Within each passage, the matched conditions used the same webpage text, embedded task and target marker; only the intended wrapper structure changed.

- I also checked the actual tokenization. In the native conditions, literal `<|im_start|>` and `<|im_end|>` strings inside the tool result became Qwen’s genuine control-token IDs, while the ordinary-text lookalikes contained no such tokens.
- Behavioural classifications were audited from the generated outputs rather than relying only on an automatic marker search. In particular, markers appearing only inside unfinished reasoning were not counted as successes, and all 320 cases were classified under a fixed rubric.
- For the causal interventions, I used **norm-matched random perturbations** to test whether changing an activation by a similar magnitude was sufficient on its own. These controls produced much smaller effects than swapping the natural opposite-role state. Self-patching also reproduced the baseline generation exactly on the tested case.

Timeline and Reflections

The initial line of investigation was to study provenance with prompt injections -- whether or not Qwen3-8B retained any residual traces which could be used to identify forged user messages as tool output. Inspired by *Prompt Injection as Role Confusion*, I trained probes to distinguish text presented as USER from text presented as TOOL. On clean Qwen3-8B examples, a frozen USER-vs-TOOL probe appeared extremely strong, reaching about 0.98 balanced accuracy and 0.997 AUROC.

This interpretation broke down when I stress-tested the probe on the nested message structures used in the actual injection setting. With native user and assistant tags, the probe began classifying genuine outer USER examples as TOOL, despite remaining highly accurate on the control. I therefore concluded that its absolute provenance scores were unreliable out-of-distribution and ended up terminating the investigation into whether prompt injection either erased or preserved formal provenance.

I therefore shifted to the more directly testable question that the rest of this document concerns.

Limitations

The experiments were designed to isolate the effect of native role structure, which means they simplify several aspects of real prompt injection.

1. **We did not show that this effect can override safety or policy constraints.** Most of our injected tasks were benign.
 - To test the stronger case, I gave Qwen access to a sandboxed mock tool with explicit warning never to call it as a result of instructions inside

tool output. Forging user input produced 0/20 forbidden calls, no different to the control. When the action was explicitly permitted, the control produced valid tool calls in 5/5 cases, ruling out the possibility that Qwen was not capable of using the tool. Thus, our results show that role forgery can significantly affect whether embedded instructions are followed or not, but we do not show that it can override an explicit higher-priority prohibition or induce harmful real-world actions.

2. All experiments were run on Qwen3-8B. Effects may depend intrinsically on this model's chat syntax, tokenizer or learned representations.
3. The activation patching results isolate an important direction, but they do not identify a specific circuit or feature or provide an unambiguous interpretation for the direction.
4. The attack depends on the text reaching Qwen's tokenizer unchanged. It's possible that other deployment frameworks or APIs sanitise text before tokenization.

Conclusions

Native Qwen user-role syntax has a strong effect on whether instructions embedded in tool output are followed. This generalises across task types and can be causally transferred by intervening on the role-token activations in early layers; furthermore, a single shared user–assistant direction also captures a substantial behaviourally relevant component. Later activation differences become large but are not by themselves sufficient to transfer behaviour.

The results suggest that Qwen3-8B rapidly processes information about native role formatting into a control signal which causally affects its response rate to embedded instructions. However, we do not claim here that this implies forged roles can override all higher-priority constraints.

References

Prompt Injection as Role Confusion (Ye et al.)

<https://role-confusion.github.io/>

ChatInject: Abusing Chat Templates for Prompt Injection in LLM Agents (Chang et al.) [\[2509.22830\] ChatInject: Abusing Chat Templates for Prompt Injection in LLM Agents](#)